

Code Review & Implementation Report

Fortune Credit Data Analysis

Introduction

This document details the specific technical and analytical improvements made to the Fortune Credit Seasonal Credit Risk Analysis project in response to a comprehensive code review. The primary focus of this refactor was to ensure analytical correctness, improve reproducibility, and align the project's conclusions strictly with the available evidence.

Summary of Refactoring Actions

Refactor Item	Review Feedback	Implementation Highlights
1. Harvest Timing Calculation	Text-ordering bugs due to string conversion prior to SQLite queries.	Retained numeric data types, handled missing values as NULL , generated labels dynamically via SQL.
2. Evidence & Conclusions	Overstated claims and missing sample sizes based on a small mock dataset.	Re-framed conclusions as illustrative observations, added sample sizes, and noted future data needs.
3. Default-Rate Chart	Mixed counts and percentages on a single Y-axis scale.	Isolated the percentage metric on the chart and placed raw counts in a separate table below.

Refactor Item	Review Feedback	Implementation Highlights
4. Repayment Metrics	"Late" loans were excluded from risk visibility.	Expanded SQL query and dashboard KPIs to report <code>Total_Late</code> and <code>Late_Rate</code> explicitly.
5. Project Reproducibility	Undeclared dependencies and syntax errors in setup documentation.	Added missing libraries to <code>requirements.txt</code> and verified environment installation steps.
6. Architecture & Quality	Duplicate cleaning code, unflagged imputed values, and unclear stack docs.	Centralized logic into <code>data_processing.py</code> , added validation checks and imputation flags.

Detailed Implementation Report

1. Fix the Harvest Timing Calculation

Review Feedback:

The `Days_to_Harvest` column was converted to text before being queried in SQLite. This caused text-ordering bugs (e.g., classifying 212 days as "0-30 days"). The reviewer requested keeping day counts numeric, representing missing values as null, and creating display labels separately.

Approach Taken

- **Numeric Preservation:** Refactored the data processing logic to ensure `Days_to_Harvest` remains a strictly numeric (`float/int`) column throughout preprocessing.
- **Null Handling:** For non-agricultural loans (such as Boda Riders and Traders) where a harvest date does not apply, the value is now explicitly set to `NaN`, which translates to `NULL` in SQLite.
- **SQL Implementation:** Display labels (e.g., `'31-90 Days'`) are now generated dynamically within the SQL `CASE WHEN` statement. The query filters using `WHERE Days_to_Harvest IS NOT NULL`, ensuring accurate numeric sorting and correct boundary classifications (30, 31, 90, 91 days).

2. Make the Conclusions Match the Evidence

Review Feedback:

The original README claimed "significantly better repayment" and an "optimal window" based on a mock dataset of only 10 loans (7 farmer loans). The reviewer requested describing the results as illustrative observations, including sample sizes, and removing claims of statistical significance.

Approach Taken

- **Revised Documentation:** Updated the README section heading to "Illustrative Observations (Proof of Concept)".
- **Sample Sizes Added:** Explicitly stated the sample sizes alongside all rates (e.g., "2/2 repaid, 100%" vs "2/5 repaid, 40%").
- **Removed Overstatements:** Removed all language suggesting statistical significance or causal effects. Added an explicit note stating that real historical data—including actual loan due dates and repayment dates—is required to evaluate if repayment truly aligns with harvest income.

3. Correct the Default-Rate Chart

Review Feedback:

The interactive chart was receiving both `Total_Loans` (counts) and `Default_Rate` (percentages), mixing two different scales on a single chart titled "Default Rate."

Approach Taken

- **Chart Isolation:** Updated the Streamlit application to plot only the `Default_Rate` percentage on the primary bar chart.
- **Separate Data Table:** Moved the raw `Total_Loans` and `Total_Defaults` counts to a clean, separate Pandas dataframe displayed directly below the chart. This ensures the visual accurately represents the percentage scale without visual distortion from count magnitudes.

4. Define the Repayment Metrics

Review Feedback:

The calculation divided defaults by all loans, but the dataset included "Late" loans. A segment could show 0% defaults while still having a significant repayment problem. The reviewer requested explaining the metrics, stating the denominator, and displaying late loans separately.

Approach Taken

- **Metric Definitions & Denominator:** Clarified that the denominator for default and late rates is total loans within the given segment. Added explicit definitions for defaulted loans (unpaid after due date) and late loans (past due but not yet defaulted) to maintain complete transparency.

- **Updated SQL Queries:** Modified the SQL aggregation logic to explicitly calculate and return `Total_Late` and `Late_Rate` alongside default figures.
- **Dashboard KPIs:** Updated the top-level Key Performance Indicators (KPIs) in the Streamlit dashboard to display four distinct metrics: Total Loans, Repaid, Late, and Defaulted. This provides a transparent view of overall portfolio health.

5. Make the Project Reproducible

Review Feedback:

`clean_data.py` required `matplotlib` and `openpyxl`, but neither was declared in `requirements.txt`. Additionally, the README's `cd` command omitted the trailing hyphen present in the repository name.

Approach Taken

- **Dependency Declaration:** Updated `requirements.txt` to include `streamlit`, `pandas`, `matplotlib`, and `openpyxl`.
- **Instruction Correction:** Corrected the "How to Run Locally" section in the README to ensure the `git clone` and `cd` commands perfectly match the actual repository name (`Fortune-Credit-Data-Analysis-`).
- **Verification:** Verified that both `streamlit run app.py` and `python clean_data.py` execute successfully in a fresh virtual environment using only the declared dependencies.

6. Improve Data Handling and Documentation

Review Feedback:

The reviewer requested sharing cleaning functions between scripts, validating data (checking for duplicates and negative amounts), flagging imputed amounts, and updating the README's description of the visualization stack.

Approach Taken

- **Centralized Processing (DRY Principle):** Created a new `data_processing.py` module. Both `app.py` and `clean_data.py` now import the shared `load_and_clean_data()` function, eliminating logic duplication.
- **Data Validation:** Added programmatic checks in the shared module for missing required columns, duplicate `Loan_IDs`, negative loan amounts, and invalid repayment statuses.
- **Imputation Flagging & Null Handling:** Added an `Imputed_Loan_Amount` boolean column to the dataframe to flag rows where the median was used to fill a missing value, maintaining complete data lineage and ensuring nulls are explicitly tracked.

- **Tech Stack Clarification:** Updated the README to accurately reflect that the live dashboard uses native Streamlit charts for interactivity, while Matplotlib is reserved for static report generation in `clean_data.py`.